

(5) Scikit-Learn ML

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, LogisticRegression,
Ridge
# from sklearn.preprocessing import OneHotEncoder
# from pandas.plotting import scatter_matrix
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_predict
from sklearn.model_selection import cross_val_score
from sklearn.metrics import mean_squared_error, auc, roc_curve
# import seaborn as sns
%matplotlib inline
```

General Pipeline + CV search

```
# The universal sklearn estimator interface
model = ModelName(param1=..., param2=...)
model.fit(X, y)
model.predict(X)

# CV
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge

model = Ridge()
param_grid = {
    "alpha": [0.01, 0.1, 1, 10, 100]
}
# This should give me valid keys for the model
model.get_params().keys()

# Wrap with GridSearch
grid = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=5, # number of folds
    scoring="neg_mean_squared_error",
    n_jobs=-1 # parallelize
)

# automatically trains for each combo
```

```

grid.fit(X, y)
best_model = grid.best_estimator_
best_params = grid.best_params_
best_score = grid.best_score_

# Example
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

param_grid = {
    "C": [0.01, 0.1, 1, 10],
    "penalty": ["l1", "l2"],
    "solver": ["liblinear"]
}

grid = GridSearchCV(
    model,
    param_grid,
    cv=5,
    scoring="accuracy"
)

grid.fit(X, y)

# Pipeline shit
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])

param_grid = {
    "model__C": [0.01, 0.1, 1, 10]
}

grid = GridSearchCV(pipe, param_grid, cv=5)
grid.fit(X, y)

```

Standardize data

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
#scaled_features = scaler.fit_transform(seeds_features)

scaled_features = pd.DataFrame(
    scaler.fit_transform(seeds_features),
    columns=seeds_features.columns,
    index=seeds_features.index
)

# Columns you want to scale
cols_to_scale = ["area", "perimeter", "compactness"]

scaler = StandardScaler()

# Fit & transform only selected columns
scaled_subset = scaler.fit_transform(seeds_features[cols_to_scale])

# Replace original columns with scaled values
seeds_features_scaled = seeds_features.copy()
seeds_features_scaled[cols_to_scale] = scaled_subset

```

Good for scatter visualization

```

fig, axs = plt.subplots(1, 3, sharey=True)
data.plot(kind='scatter', x='TV', y='sales', ax=axs[0], figsize=(16, 5),
grid=True)
data.plot(kind='scatter', x='radio', y='sales', ax=axs[1], grid=True)
data.plot(kind='scatter', x='newspaper', y='sales', ax=axs[2], grid=True)
plt.show()

```

Take the data and train

```

feature_cols = ['TV', 'radio', 'newspaper']
X = data[feature_cols]
y = data.sales

X.describe()

lin_reg = LinearRegression() # create the model
lin_reg.fit(X, y) # train it

```

```

for f in range(len(feature_cols)):
    print("{0} * {1} + ".format(lin_reg.coef_[f], feature_cols[f]))
print(lin_reg.intercept_)

0.0457646454553976 * TV +
0.18853001691820448 * radio +
-0.0010374930424763285 * newspaper +
2.9388893694594103

```

Predict and plot prev values

```

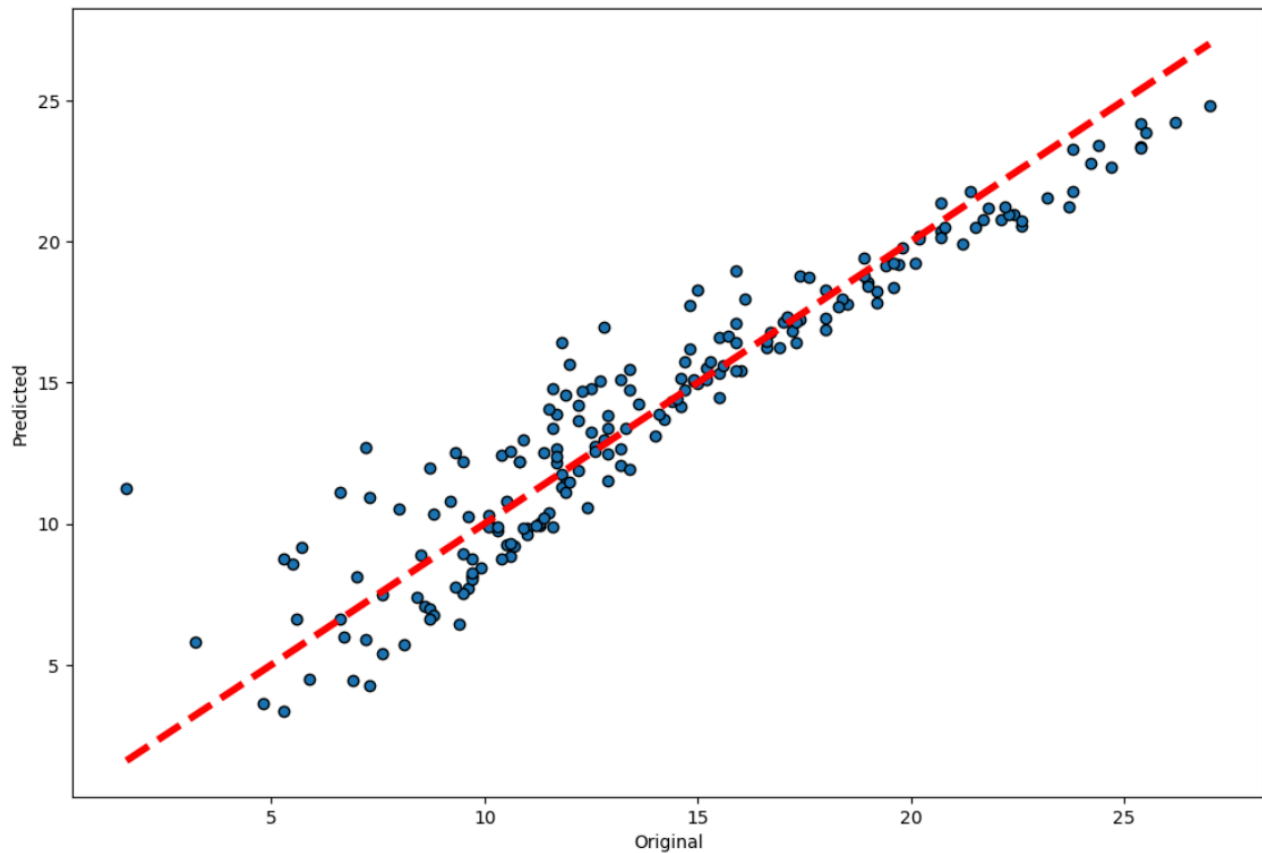
lr = LinearRegression()

# cross_val_predict returns an array of the same size as `y` where each
entry
# is a prediction obtained by cross validation:
predicted = cross_val_predict(lr, X, y, cv=5)

# Plot the results
fig, ax = plt.subplots(figsize=(12, 8))
ax.scatter(y, predicted, edgecolors=(0, 0, 0))
ax.plot([min(y), max(y)], [min(y), max(y)], 'r--', lw=4)
ax.set_xlabel('Original')
ax.set_ylabel('Predicted')
plt.show()

mean_squared_error(y, predicted)
# 3.0729465971002106

```



```
ridge = Ridge(alpha=6) # (alpha is lambda)

# cross_val_predict returns an array of the same size as `y` where each
entry
# is a prediction obtained by cross validation:
predicted_r = cross_val_predict(ridge, X, y, cv=5)
```

What are the **features**?

- name: Name of the passenger
- sex: Male or Female
- age: Age in years
- sibsp: # of siblings / spouses aboard the Titanic
- parch: # of parents / children aboard the Titanic
- ticket: Ticket number
- fare: Ticket price
- cabin: Cabin number
- embarked: Port of Embarkation

What is the **response**?

- survived: whether the passenger survived the disaster or not

```

titanic_features = ['sex', 'age', 'sibsp', 'parch', 'fare']

pd.get_dummies(
    data,
    prefix=None,
    prefix_sep="_",
    dummy_na=False,
    columns=None,
    drop_first=False,
    dtype=None
)
# only data and columns seem interesting, actually maybe dtype as well

# The features vector
X = pd.get_dummies(titanic[titanic_features])
X.head()

len(X[X.isna().any(axis=1)])
# 264

X = X.fillna(X.mean())
len(X[X.isna().any(axis=1)])
# 0

y = titanic['survived']
logistic = LogisticRegression(solver='lbfgs')

from sklearn.model_selection import cross_validate

scores = cross_validate(
    logistic,
    X,
    y,
    cv=10,
    scoring=["precision", "recall"],
    return_train_score=False
)

print("Precision:", scores["test_precision"].mean())
print("Recall:", scores["test_recall"].mean())

```

	age	sibsp	parch	fare	sex_female	sex_male
0	29.0000	0	0	211.3375	True	False
1	0.9167	1	2	151.5500	False	True
2	2.0000	1	2	151.5500	True	False
3	30.0000	1	2	151.5500	False	True
4	25.0000	1	2	151.5500	True	False

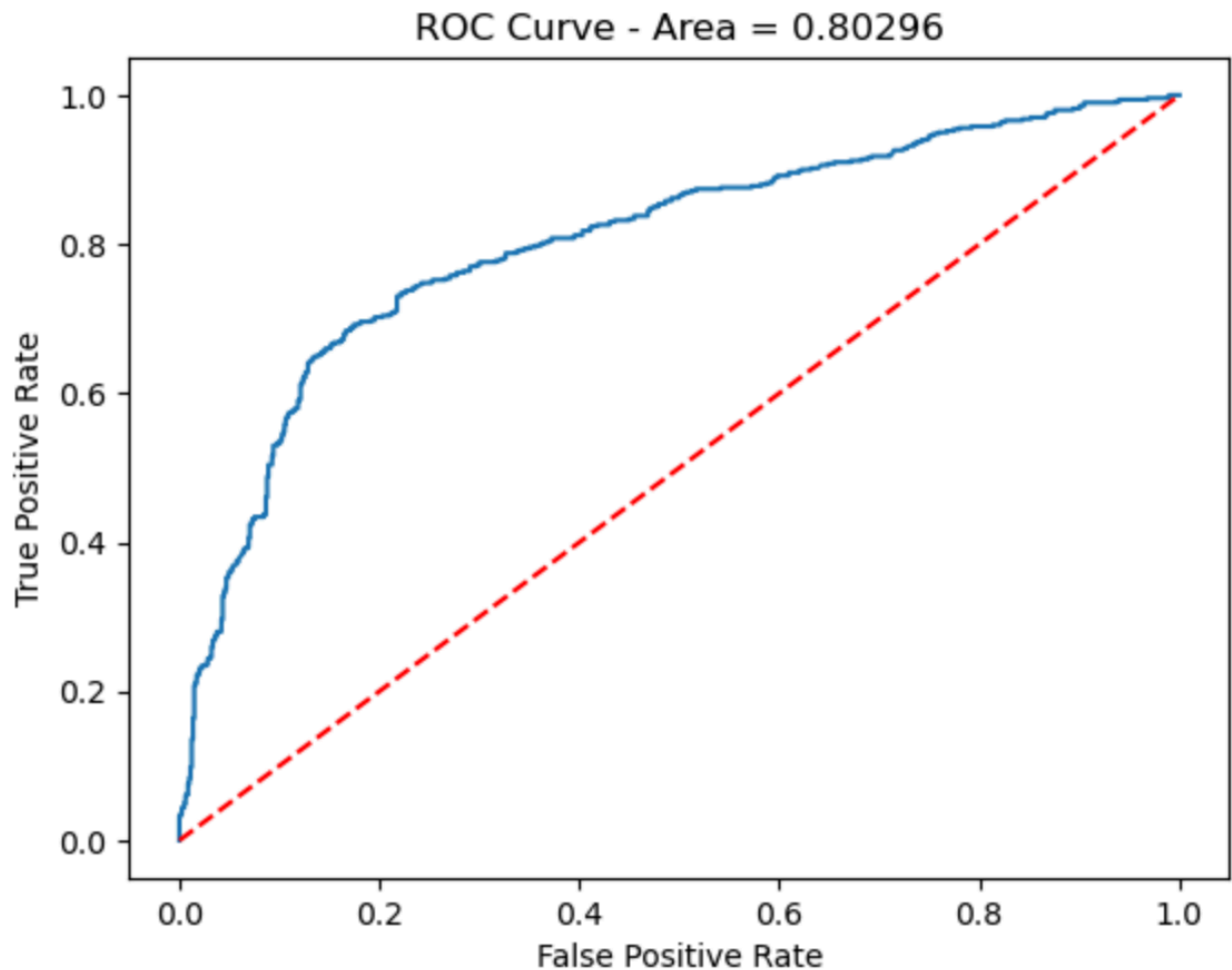
```

cv=5          # integer
cv=KFold(5)
cv=StratifiedKFold(5) # For classification

# Predict the probabilities with a cross validationn
y_pred = cross_val_predict(logistic, X, y, cv=10, method="predict_proba")
# Compute the False Positive Rate and True Positive Rate
fpr, tpr, _ = roc_curve(y, y_pred[:, 1])
# Compute the area under the fpt-tpf curve
auc_score = auc(fpr, tpr)

plt.plot(fpr, tpr)
plt.plot([0, 1], [0, 1], 'r--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Area = {:.5f}".format(auc_score));
plt.show()

```



Predict some points

```
# Index(['age', 'sibsp', 'parch', 'fare', 'sex_female', 'sex_male'],
dtype='object')

test = [25, 0, 0, 100, 0, 1]
"YES" if logistic.predict([test])[0] > 0 else "NO"

logistic.predict_proba([test])
array([[0.5528599, 0.4471401]])

test = [35, 0, 0, 100, 1, 0]
"YES" if logistic.predict([test])[0] > 0 else "NO"

print(logistic.predict_proba([test])[0])
[0.11464109 0.88535891]
```

For clusters (might be interesting to plow cluster centers)

```
plt.scatter(X[:,0], X[:,1], alpha=0.6)
plt.xlabel("X")
```



```
plt.ylabel("Y")
plt.title("Artificial clusters (%s samples)" % total_samples)

for c in centers:
    plt.scatter(c[0], c[1], marker="+", color="red")
```

Cluster Seeds

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.manifold import TSNE

# =====
# Question 1.1 – Dataset preparation
# =====

# Keep only meaningful numerical features
# Remove ID and the true label (seedType) since we perform clustering
seeds_features = seeds.drop(columns=["ID", "seedType"])

# -----
# Visualize feature distributions
# -----

feature_names = seeds_features.columns

fig, axes = plt.subplots(1, len(feature_names), figsize=(14, 2),
sharey=False)

for i, feature in enumerate(feature_names):
    axes[i].hist(seeds_features[feature], bins=20, alpha=0.7)
    axes[i].set_title(feature)

plt.suptitle("Feature Distributions")
plt.tight_layout()
plt.show()

# -----
# Feature scaling (important for distance-based algorithms)
# -----

# Standardize features to mean=0 and variance=1
scaler = StandardScaler()
scaled_features = scaler.fit_transform(seeds_features)

print("Example of scaled feature vector:")
```

```

print(scaled_features[0])

# =====
# Question 1.2 – K-Means clustering (Elbow Method)
# =====

def plot_elbow_method(X, k_min=2, k_max=11):
    """
    Computes and plots the Sum of Squared Errors (SSE)
    for different values of K using K-Means.
    """
    sse = []

    for k in range(k_min, k_max):
        kmeans = KMeans(n_clusters=k, random_state=10)
        kmeans.fit(X)
        sse.append(kmeans.inertia_)

    plt.figure(figsize=(6, 4))
    plt.plot(range(k_min, k_max), sse, marker='o')
    plt.xlabel("Number of Clusters (K)")
    plt.ylabel("Sum of Squared Errors (SSE)")
    plt.title("Elbow Method for Optimal K")
    plt.grid(True)
    plt.show()

# Apply the elbow method
plot_elbow_method(scaled_features)

# =====
# Question 1.3 – Cluster visualization with t-SNE
# =====

# Reduce dimensionality to 2D for visualization
tsne = TSNE(
    n_components=2,
    init='random',
    learning_rate='auto',
    random_state=0
)

X_tsne = tsne.fit_transform(scaled_features)

# Perform K-Means clustering with chosen K
kmeans = KMeans(n_clusters=3, random_state=0)
cluster_labels = kmeans.fit_predict(scaled_features)

# -----
# Side-by-side comparison: True labels vs K-Means clusters
# -----

```

```

fig, axes = plt.subplots(1, 2, figsize=(8, 3), sharey=True)

# True labels
axes[0].scatter(
    X_tsne[:, 0],
    X_tsne[:, 1],
    c=seeds["seedType"],
    alpha=0.7
)
axes[0].set_title("Original Seed Types")

# Discovered clusters
axes[1].scatter(
    X_tsne[:, 0],
    X_tsne[:, 1],
    c=cluster_labels,
    alpha=0.7
)
axes[1].set_title("K-Means Discovered Clusters")

plt.suptitle("t-SNE Visualization of Seed Clusters")
plt.tight_layout()
plt.show()

```

Available in scikit

LogisticRegression

KNeighborsClassifier

SVC

DecisionTreeClassifier

RandomForestClassifier

GradientBoostingClassifier

AdaBoostClassifier

GaussianNB

LinearRegression

Ridge

Lasso

ElasticNet

SVR

DecisionTreeRegressor

RandomForestRegressor
GradientBoostingRegressor

KMeans
DBSCAN

AgglomerativeClustering